Q1. 什么才算完成（300 字以内）

1. 开发前先和产品明确需求细节：

批量生成失败后的扣费点数如何回退？

定时发布异常后的处理逻辑是怎样的？

定时发布的时间是以本地时间为基准？还是以一个绝对时间为基准？切换时区后的发布时间是否需要同步变动？

指标中美元与港币的 7.8 汇率是否是固定的，需要取实时汇率吗？

对于效果报表：支付后多久未退款的订单才能被纳入报表统计中？

2. 再和开发确认预期的实现逻辑：

本次迭代中新功能的影响范围是怎样的，有哪些业务需要回归。

批量生成贴文过程中，失败后的重试机制如何实现，是 MQ 异步重试还是定时任务轮询重试

3. 上线的定义：

主流程无高优 bug

订单数据对账后无异常

批量生成功能压测通过

所有异常情况都有对应的兜底方案（业务处理超时、三方回调异常、有明确的版本回滚方案、产品侧功能验收通过）

Q2. 能不能上线（300 字以内，请直接写出你发到项目群里的结论）

目前测试剩余 3 个问题待修复：

1. 批量生成 10 个商品时，大约每 10 次会有 1 条超时失败，用户需要手动重试（p1）

2. 美西时区的商家在每月 1 号当天，首页“本月 AI 带来收入”显示的是上个月的数据（**p0**）

3. 商家修改账号时区后，已经排期的贴文发布时间没有跟着调整（p1）

结论：

问题 1 和 3 可以遗留，问题 2 需修复后才可上线

问题 1 有重试兜底逻辑，复现概率较低，不会造成流程阻塞，可以遗留。上线后需要添加失败率监测机制，灰度过程中需要重点观察。

问题 2 为特定用户必现问题，直接影响到报表的账目核算，必须要修复后才可上线。

问题 3 为时区计算异常，该场景为小概率场景，可以遗留。建议先针对该场景添加相关提示，后续版本再进行优化。

Q3. AI 扫描结果（400 字以内，可用表格）

开发用 AI 工具扫描了 v2.3，得到以下 12 条问题：

1. 定时发布：排期时间设成过去的时间，系统没有拦截，贴文被立即发布

2. “Generate” 按钮在 320px 宽的屏幕上文字被截断

3. 浏览器控制台出现 React 警告：列表 key 重复

4. 批量生成 10 个商品时，第 10 个商品的图片偶尔没有带上

5. 报表页加载耗时 3.2 秒，超过“建议值 2 秒”

6. 接口 /api/posts 的返回数据中包含商家社媒账号的 access_token

7. 生成的英文文案中，colour 和 color 两种拼写混用

8. 生成的贴文中，偶尔使用了商品原价，而不是当前的促销价

9. 代码中有未使用的变量

10. 定时发布成功后，用户没有收到任何通知

11. 排期时可以选择昨天的日期

12. 登录页的密码输入框没有“显示密码”按钮

你会把哪些提给开发？优先级怎么排？哪些不提，或者换一种方式处理？为什么？

答：

问题 1 和问题 11 合并提交 P1 优先级 bug。均属于时间校验缺失，影响范围较大。

问题 2 提交 P2 优先级 bug，属于兼容性问题。

问题 3 提交 P2 优先级 bug，业务影响较小，用户端大概率无感知，可能会影响到某些排序和查询逻辑。

问题 4 提交 P1 优先级 bug，属于核心功能异常。

问题 5 提交 P2 优先级 bug，属于性能优化问题，影响用户体验。

问题 6 提交 P0 优先级 bug，token 直接暴露在响应中属于重大安全风险。

问题 7 提交 P2 优先级 bug，该问题属于 prompt 优化问题，必要时可以转需求，走需求去统一规范 AI 生成的术语，建议 prompt 增加文案内容的检测步骤。

问题 8 提交 P1 优先级 bug，金额问题需要提高优先级，该 bug 会导致商家产生虚假宣传的合规问题。

问题 9 不提交 bug，直接转需求分配给开发统一优化处理。

问题 10 应该先确认需求文档，需求文档中若有该场景的描述就直接提交 P1 优先级 bug，属于业务实现缺失。若没有需求文档则需通知产品先补充需求，再由产品提优化单交由开发处理。

问题 11 已和问题 1 合并提单

问题 12 应该先确认需求文档，需求文档中若有该场景的描述就直接提交 P1 优先级 bug，属于业务实现缺失。若没有需求文档则需通知产品先补充需求，再由产品提优化单交由开发处理。

Q4. 你的 AI 工作流（300 字以内）

之前有使用 coze 做过一个根据需求文档自动生成测试用例的工作流。在配置的过程中产生了很多问题。

比如需求文档中如果包含图片，但是当前配置的模型又不支持图片识别，就会出现用例覆盖范围不足的情况。解决方案是为工作流中的每个步骤单独配置模型，交由效果最好的模型实现。

再就是会遇到需求文档过长的情况，这种情况会导致上下文过长，从而影响到测试用例生成的质量。解决方案是将文档拆分成多个模块逐一生成。注意每次都要将该需求的总体概况与单独模块一并发给 AI，能最大程度避免 AI 理解偏差的问题。

人工检查用例生成质量时，发现一些用例没法对应到具体需求中，属于 AI 幻觉产生的无效用例，解决方案是要求 AI 对每条用例都增加一列，用来标注用例在需求中具体的来源。极大的提高了人工查漏补缺的效率。

Q5. 数字对不上（300 字以内）

排查前应该先了解清楚用户的问题，明确用户所说的两个金额来源是哪里，确认是我方业务问题后再开始结合日志进行排查。

先排查金额的时间统计区间是否正确，一般在服务端日志中会有对应的统计 sql，排除 sql 中时区边界填充不对的问题。

再就是查同一条贴文是否有被重复统计的问题。比如后续用户手动编辑了一遍已有贴文，导致贴文的唯一标识变了，有可能统计时会将编辑前和编辑后贴文的数据都算进去。

再逐一排查该统计金额所涉及的每笔订单的订单状态，可能存在某些异常订单被统计的bug。

要是最后实在没有查明具体原因，可以结合当前用户上报的埋点分析一下用户行为，也可以发现一些异常的场景。

Q6. 产品建议（300 字以内）

1. 批量生成过程中可以支持即时展示已生成好的商品贴文，支持用户即时去修改某一条贴文，无需用户等待所有商品贴文生成后再进行操作，尽量缩短用户的时间成本，提高用户体验。
2. 定时发布后需要及时将发布结果触达给用户，例如站内信或者短信的形式。避免用户在经过较长时间后才发现发布失败，从而产生客诉。
3. 报表可以增加收入饼图，每个扇形支持展示对应贴文，方便商家评估贴文效果。

Q7. 已修复，但还在出现（300 字以内）

先确定问题的场景，确认该问题和已修复的是不是同一个问题。是的话就先获取问题用户使用的版本，确保对应版本中已经包含了该问题的修复。

如果不是版本问题，那就继续排查用户的日志。找到具体的复现路径，可能该问题和已知问题的复现路径不一致。若排查后发现不是一个问题，需要先给用户提供临时解决方案，然后给开发提缺陷单走 hotfix 修复。

事后再进行问题复盘，补充相应的 P0 回归用例，同时找开发增加对应场景的告警，确保问题后续不再发生。

Q8. 是 bug 还是需求（300 字以内）

先找产品明确该问题是否需要修复，修就转需求单提给产品，同时要求产品补充完整prd，补充完整之后再流转至开发进行实现。

一般我会将明确与 prd 中实现不一致的问题提交为 bug。其他类型的问题都会先和产品进行确认，需要实现的直接走需求单，不会在不经过确认的情况下单独给开发提 bug 单。

Q9. 主动推动（300 字以内）

我一般在业务测试过程中会主动写一些脚本去自动核对数据。

在一次会员订单支付需求上线后，我收到脚本执行异常的通知，具体表现为部分订单的支付金额和实际给用户下发的权益对不上。

事后排查后发现是运营人员在上线后添加了一个测试商品，将金额改为了 0.1，测试完成之后只是在后台将该商品做了隐藏，并没有操作下架。

这就导致某些懂技术的用户通过批量调用创建订单接口发现了这个商品 id。

由于该问题是在上线后第二天发现的，索性没有造成重大经济损失，事后统计日均损失金额在 1000 元左右。

Q10. 你的经历（改变产品设计，300 字以内）

在 AI 智能客服 RAG 系统的测试过程中，我用 AI 去收集了客服群中用户反馈的所有问题，按照频率对数据做了清洗和排序。发现了很多召回文档中没有编写到的问题。这些问题会导致智能客服没法召回相关问题的片段，从而导致 AI 幻觉的情况。

比如用户在访问分享链接之后，如果主域名失效了，业务逻辑是会重定向到一个纯数字域名来保证用户能够正常访问。但是访问者会怀疑该链接是不是官方链接，进而去询问智能客服。

例如这种非业务的场景大多都来源于我用脚本收集的真实问题，后续也是影响到了产品侧的工作流程，产品每次编写新功能的召回文档时，都需要先收集真实用户问题做补充，而非单纯针对业务功能做编写。